

实 验 报 告

一、实验基本信息

实验名称：深度学习 LeNet 网络识别 Mnist 手写数字集

姓名：赵宸

学号：2302020530

班级：化学 235

实验日期：2025/7/22

实验平台：Pycharm/Conda

二、实验目的

了解 LeNet 的基本构造和工作原理

了解卷积层的工作原理，并比较其和全连接层的不同点

练习数据集的读取，切分等基本操作

学会训练深度学习网络，并用交叉熵损失函数展示模型效果

掌握 numpy, pandas 等基本第三方库的用法，掌握基本的模型效果评价指标

三、实验环境配置

Pycharm/Conda 解释器，集成的 3.12 版本的 Python

四、实验步骤与过程记录

实验步骤

导入必要的第三方库-读取数据集并进行图像重构-搭建网络-训练网络-定义交

叉熵损失函数开始迭代-读取测试集并进行测试-可视化预测结果

实验源代码（未修改任何参数）：

```
import os

os.environ['KMP_DUPLICATE_LIB_OK']='TRUE'

import numpy as np

import pandas as pd

import torch
```

```

from torch.utils.data import DataLoader, TensorDataset

import torch.nn as nn

import torch.nn.functional as F

import torch.optim as optim

import matplotlib.pyplot as plt


train_data=pd.read_csv("C:/Users/HP/Desktop/Data/train.csv")
train_images=train_data.iloc[:,1:].values.astype(np.float32)/255.0
train_labels=train_data.iloc[:,0].values.astype(np.int64)
train_images=torch.tensor(train_images).view(-1,1,28,28)
train_labels=torch.tensor(train_labels)
# print(train_images.shape, train_labels.shape)


batch_size=128

train_dataset=TensorDataset(train_images, train_labels)
train_loader=DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

# for imgs, labels in train_loader:
#     print(imgs.shape, labels.shape)
#     break

class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.conv1=nn.Conv2d(1, 6, kernel_size=5)
        self.pool1=nn.AvgPool2d(kernel_size=2, stride=2)
        self.conv2=nn.Conv2d(6, 16, kernel_size=5)
        self.pool2=nn.AvgPool2d(kernel_size=2, stride=2)
        self.fc1=nn.Linear(16*4*4, 120)
        self.fc2=nn.Linear(120, 84)

```

```

        self.fc3=nn.Linear(84, 10)

    def forward(self, x):
        x=self.pool1(F.relu(self.conv1(x)))
        x=self.pool2(F.relu(self.conv2(x)))
        x=x.view(-1, 16*4*4)
        x=F.relu(self.fc1(x))
        x=F.relu(self.fc2(x))
        x=self.fc3(x)

        return x

# model=LeNet()
# print(model)
model=LeNet()
criterion=nn.CrossEntropyLoss()
optimizer=optim.Adam(model.parameters(), lr=0.001)
num_epochs=10
for epoch in range(num_epochs):
    model.train()
    running_loss=0.0
    for images, labels in train_loader:
        optimizer.zero_grad()
        outputs=model(images)
        loss=criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss+=loss.item()

    print(f'Epoch[{epoch+1}/{num_epochs}], Loss: {running_loss/len(train_loader):.4f}')

```

```

test_data=pd.read_csv('C:/Users/HP/Desktop/Data/test.csv')
test_images=test_data.values.astype(np.float32)/255.0
test_images=torch.tensor(test_images).view(-1,1,28,28)
test_loader=DataLoader(test_images,batch_size=batch_size,shuffle=False)

model.eval()
predictions=[]
with torch.no_grad():
    for images in test_loader:
        outputs=model(images)
        _,predicted=torch.max(outputs,1)
        predictions.extend(predicted.numpy())

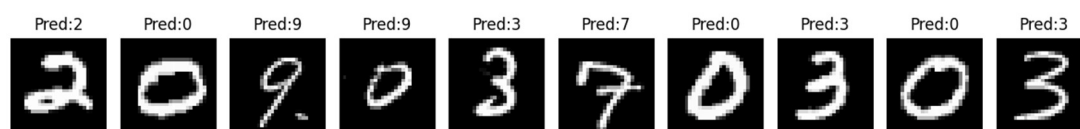
plt.figure(figsize=(12,4))
for i in range(10):
    plt.subplot(2,10,i+1)
    plt.imshow(test_images[i].squeeze().numpy(),cmap='gray')
    plt.title(f'Pred: {predictions[i]}')
    plt.axis('off')
plt.tight_layout()
plt.show()

```

然后在此基础上，对部分参数进行修改，将平均池化改成了最大池化，效果比原来要好，数据结果将在第五部分进行分析

五、实验结果与分析

按照原网络搭载，没有修改任何参数的情况下，结果如下：



```
Epoch[1/10], Loss:0.5743
Epoch[2/10], Loss:0.1927
Epoch[3/10], Loss:0.1218
Epoch[4/10], Loss:0.0926
Epoch[5/10], Loss:0.0746
Epoch[6/10], Loss:0.0640
Epoch[7/10], Loss:0.0563
Epoch[8/10], Loss:0.0481
Epoch[9/10], Loss:0.0452
Epoch[10/10], Loss:0.0403
```

可以看到，交叉熵的损失下降还是比较明显的，单从交叉熵损失函数来看，模型比较成功，通过可视化情况，10个里面的准确率是90%，第四张图像，可能由于左上角的部分没有完全连接起来，导致了这张图像的识别比较困难。但是也不排除过拟合的情况，由于图像没有标签，所以没法输出准确率，而且无法评估模型的泛化能力，而且Mnist手写数字的数据集规模比较小，所以如果对于大批量的数据，模型的泛化能力不一定特别强。

修改了池化层之后，又进行了一次模型实验，发现刚才无法识别的第四章图片有了提升，可以正确的进行识别了

```
Epoch[1/10], Loss:0.5125
Epoch[2/10], Loss:0.1296
Epoch[3/10], Loss:0.0884
Epoch[4/10], Loss:0.0679
Epoch[5/10], Loss:0.0541
Epoch[6/10], Loss:0.0453
Epoch[7/10], Loss:0.0391
Epoch[8/10], Loss:0.0346
Epoch[9/10], Loss:0.0335
Epoch[10/10], Loss:0.0278
```



分析下来，很有可能是因为之前 0 无法识别是因为左上角的连续性不够强，平均池化并没有使这个地方有所改善，但是最大池化使得部分黑色区域被变化成白色区域，使得数字的线条轮廓更宽，连续性更强，更容易识别到这里的特征，所以这也就是为什么从平均池化变化成最大池化之后，这里的输出结果有了一个很不错的提升

六、总结

思考题 1：卷积层与全连接层的参数量有何差异？

全连接层的话是需要每一个神经元全部进行链接，比如说输入图像为单通道 10×10 像素，输出为 10 的话，参数量就是 $10 \times 10 \times 10 = 1000$ 个。但是卷积层，由于它可以设置步长，卷积核的大小等等，使得原本的图像会损失一部分信息，也就是丢失一部分像素，也就是提取图像特征的过程，所以自然所需要的参数量也会小很多。例如，如果对于同一幅图像，输出和输入一样，卷积核 3×3 ，步长为 1 的情况下，对于 $224 \times 224 \times 3$ 的图像，链接到 1000 维输出，全连接的参数量约为 $224 \times 224 \times 3 \times 1000 = 1.5$ 亿，若卷积输出 64 通道，则参数量仅为 $3 \times 3 \times 3 \times 64 + 64 = 1792$ ，相差可以达到 8 万倍

思考题 2：为什么卷积层的模型效果会更好？

卷积核在工作的过程中，关注的是它自身窗口大小所能扫描到的局部区域，图像中相邻像素的关联性远高于远距离的像素，固定大小的卷积核使得图像在这些内容上的注意力更集中，自然提取到的特征也会更好；前面也提到过。卷积层使得模型参数量大幅度下降，并且更好的防止了过拟合现象的发生，让模型训练能力提取特征学习的能力更强，而非记住数据集的能力更强；一般卷积核都有几个通道，每一个通道可以学习特定的特征，分工明确，使得学习到的特征更加清晰；卷积核是在二维平面上进行滑动，不会损失图像原有的结构，但是全连接层必须把图像展平为一维向量，这势必会导致更多信息的丢失

思考题 3：平均池化与最大池化的对比与优劣分析

平均池化是保留窗口的像素平均值，这可以保留更多的全局信息，更加平滑，对区域的整体趋势更加敏感，但是同时他对噪声也很敏感，因为噪声一定会被平均到结果当中，可能会弱化一些重要特征的响应。最大池化呢则是可以有效保留局部特征的强度，一些很突出的特征，颜色非常明亮的，边缘，角点等就可以被突出出来，但是由于采用的是最大值，无法兼顾其他值，很大程度上会丢失部分的细节信息，可能会过度简化特征信息，不过最大池化的参数量更小，计算效率会更快。对于图像识别类任务（需要捕捉显著特征），优先用最大池化，但是对于语义分割的任务，平均池化可能会更适合

思考题 4：为什么需要 reshape 成 $(N, 1, 28, 28)$

是为了适配卷积层的输入格式，并且符合图像数据的内在结构， 28×28 是网络的图像输入的要求，1 是指 LeNet 只能处理灰度图像，所以通道必须为 1。所以 reshape 是为了确保卷积操作能正确提取图像的局部特征，如果不进行 reshape，扁平化的向量会丢失空间信息，导致卷积层无法正常工作。

总结下来，通过这次实验，学会了 LeNet 的基本架构，掌握了其卷积池化的基本操作，并且用代码进行了实验，并修改了一些参数和操作来优化模型，都取得了不错的效果，对该网络有了一个较为全面的认识，并巩固了一些模型评价指标和数据集的基本操作